Revision Note (2026-07-09)
Table of Contents
Chapter 1 — Introduction
Chapter 2 — Project Description and Goals
Chapter 3 — Technical Specification
    3.4.1 Update — Risk Register Addendum
Chapter 4 — Design Approach and Details
Chapter 5 — Methodology and Testing
    5.1 Module Description
    5.2 Testing — *updated*
Chapter 6 — Project Demonstration
    6.3 Demonstration Walkthrough — Live Screenshots
Chapter 7 — Result and Discussion
    7.1 India WDN Results: HARP vs. Dinic's — *updated*
    7.5 GAT Pressure Surrogate Accuracy — *updated, partially*
Chapter 8 — Conclusion
    8.1 Future Scope — *addendum*
Chapter 9 — References
Appendix A — Sample Code (Addendum)

HARP-WDN: HYDRAULICS-AWARE RESILIENT PRIORITIZATION FOR WATER DISTRIBUTION NETWORKS USING GRAPH ATTENTION NETWORKS AND IoT-DRIVEN REAL-TIME RECOVERY

B.Tech. BCSE497J – Project-I Reg. No. 23BKT0040 — Anushka Nair Reg. No. 23BCE0065 — Anjali Under the Supervision of Prof. G.M. Karthik School of Computer Science and Engineering (SCOPE), VIT Vellore July 2026

Revision Note (2026-07-09)

This is a revised edition of the Project-I report, updated against the current state of the `WDS` repository (commit `dd1cac0`, "Add India WDN geo module, anomaly evaluation, and API tests", on top of `c5fd46d`). The original submission described the system as it existed at `c5fd46d`. Since then the project has moved forward in ways that change several factual claims in Chapters 3, 5, 6 and 7. In keeping with the report's own stated commitment to "scientific honesty over marketing" (Abstract, p. xii), this revision:

- Re-verifies every quantitative claim in Chapters 5 and 7 against the current codebase and a live run of the system, rather than silently carrying old numbers forward.
- Documents four real bugs that were found and fixed since the original submission (Section 5.2.1), because they materially affected the correctness of numbers the original report presented as validated.
- Replaces the narrative-only Chapter 6 demonstration with screenshots captured from an actual running instance of the frontend and backend (`docs/screenshots/`), driven end-to-end with Playwright rather than described from memory.
- Flags, rather than papers over, one metric (L-Town GAT accuracy) that could not be freshly re-verified in this revision because the upstream summary file was overwritten by a newer run for a different network.

Sections not touched by the underlying code changes (Chapters 1, 2, 4, the Hazen–Williams hydraulics in 5.1.2, Sections 7.2–7.3 and 7.6–7.7, Chapter 9 References) are carried forward unchanged from the original submission and are not reproduced with edit markers.

Table of Contents

1. Introduction
2. Project Description and Goals
3. Technical Specification
4. Design Approach and Details
5. Methodology and Testing
6. Project Demonstration
7. Result and Discussion
8. Conclusion
9. References Appendix A – Sample Code

Chapter 1 — Introduction

(Unchanged from the original submission — Background, Motivations and Scope of the Project. See Section 1.3 for the ten-city catalogue, which remains Bengaluru, Delhi, Mumbai, Hyderabad, Chennai, Pune, Kolkata, Jaipur, Ahmedabad and Lucknow — verified still current in `backend/geo/india_wdn.py:INDIA_CITIES`*.)*

The core thesis is unchanged: the authors' earlier IEEE conference contribution proposed Dinic's algorithm and Capacity Scaling coupled with IoT sensing for water-network recovery, but never derived pipe capacity from real hydraulics. HARP-WDN repairs that gap by embedding a Hazen–Williams feasibility test and priority-weighted scoring inside a modified Dinic's blocking-flow search. Everything in this revision reflects further hardening of that same system, not a change of thesis.

Chapter 2 — Project Description and Goals

(Unchanged — Literature Review, Research Gap, Objectives, Problem Statement and Project Plan. None of the reviewed literature or the stated objectives changed; only the implementation maturity underneath them did.)

Chapter 3 — Technical Specification

Sections 3.1–3.3 (Requirements, Feasibility Study, System Specification) are unchanged. Two updates to Section 3.4 (Risk Analysis):

3.4.1 Update — Risk Register Addendum

RISK	LIKELIHOOD	IMPACT	MITIGATION TAKEN
Integration-layer bugs invisible to unit tests (algorithms tested only against mocked/tiny graphs, never through the live API)	High (materialised)	High	Added `backend/tests/test_api.py`, a `TestClient`-based integration suite that exercises `/simulate/scenario` end-to-end against a real network. Caught two silent-failure bugs (Section 5.2.1) that 15 passing unit tests had not surfaced.
Graph-theoretic capacity proxy (diameter×10) systematically over-stating deliverable flow on the India WDN layer's realistic pipe diameters	Medium (materialised)	Medium	Replaced with a velocity-based capacity (`v_max=1.5 m/s × cross-sectional area` , per AWWA M31 design-velocity guidance) specifically for the India WDN layer, after measuring the old proxy was 20–85× too generous at the 0.10–0.40 m diameters this generator uses. The Net3/Hanoi/L-Town benchmark layer keeps diameter×10 as its default, unchanged, so Chapter 7's benchmark-layer numbers are unaffected.
Anomaly-detector evaluation only exercised failure scenarios, so precision was 1.0 "by construction" — a methodologically invalid measurement, not evidence of an accurate detector	Medium (materialised)	Medium	`evaluate_detector` now runs a negative (no-failure) class of equal size to the positive class, so false positives and true negatives can be counted at all. See Section 7.1.3 for the resulting, considerably more honest numbers.

Chapter 4 — Design Approach and Details

(Unchanged. The three-layer architecture, DFDs, use-case diagram, class diagram and sequence diagram in the original Figures 4.1–4.6 remain structurally accurate. The HARP class still consults a pressure oracle and runs a Hazen–Williams feasibility check inside a modified Dinic's search; what changed is what feeds that oracle on the India WDN layer — see Section 5.1.4 below — not the architecture itself.)

Chapter 5 — Methodology and Testing

5.1 Module Description

5.1.1 India WDN Generator (`backend/geo/india_wdn.py`) — *updated*

Two corrections to the original description, plus one new capability that did not exist at the time of the original submission:

1. **The MST backbone uses Prim's algorithm, not Kruskal's.** `_connect_nodes()` grows the tree outward from the first source node using a Prim's-style nearest-unattached-node search over a haversine distance matrix (`scipy.spatial.distance.cdist`), not Kruskal's algorithm with union–find as the original report stated. The redundant -edge pass that follows it — biased toward hospitals (priority bonus 0.9), fire stations (0.85) and schools (0.75), decreasing for lower -priority types — is unchanged and matches the original description exactly.

2. **A network is now optionally built from real OpenStreetMap points of interest via OSMnx**, not only from the deterministic synthetic template. `fetch_city_pois()` first tries `osmnx.features_from_point()` for hospitals, fire stations, schools, colleges, parks and markets within each city's configured radius; on any failure (package not installed, network unreachable, empty result) it falls back to the fully deterministic synthetic template described in the original report — the same guaranteed minimums (2 hospitals, 2 fire stations, 3 schools, 1 college, 2 markets, 2 parks) and the same MST-plus -redundant-edges construction apply to both paths. This means the exact topology returned for a given city can now vary between environments depending on OSMnx availability; the cached, seeded synthetic fallback remains the reproducible path used for the results in this report.

3. **Pipe class assignment and Hazen–Williams parameters are unchanged** (main 0.40 m/C=130, secondary 0.25 m/C=120, local 0.15 m/C=110, branch 0.10 m/C=100), as is the cache filename convention (`india_wdn_{city}_n{n}.json`).

5.1.3 Standard Dinic's Baseline (`backend/algorithms/dinic.py` / `backend/geo/india_wdn.py:run_dinic_simulation`) — *updated*

The original report's central claim — that Dinic's assigns capacity as `pipe_diameter × 10`, a crude proxy carried over unchanged from the conference paper specifically so HARP's improvement over it could be measured honestly — is **still true of the Net3/Hanoi/L-Town benchmark layer** (`Dinic.run()` still defaults to `diameter × 10` whenever a caller does not supply an explicit `capacity` edge attribute; Chapter 7.4–7.7 is unaffected by anything in this section).

It is **no longer true of the India WDN layer**. Two changes there:

- **Capacity model.** `run_dinic_simulation()` now supplies each edge with an explicit `capacity = v_max × (π/4) × D²`, using a design velocity `v_max = 1.5 m/s` — the top of the ~0.6–1.5 m/s range AWWA Manual M31 cites as normal distribution-main design velocity. This was a direct response to measuring that `diameter × 10` was **20–85× more generous** than this realistic capacity at the 0.10–0.40 m diameters the India WDN generator actually uses; Dinic's decisions on this layer were being computed against a capacity ceiling that had no physical grounding at all, not merely one that ignored head loss.
- **The reported number is no longer the max-flow ratio itself.** Dinic's own max-flow allocation is now used only to set each junction's *demand target*; the network is then handed to a real WNTR/EPANET pressure-driven-demand steady-state solve with the failed pipe(s) closed (and, for a burst scenario, modelled as an actively discharging emitter at the break point, not merely an idealised already-isolated outage). Demand

> Served, Critical Node Service and PVC are computed from that real hydraulic solve's actual pressures and delivered flow — the same evaluation path HARP's own results already went through — not from the graph-only ratio. This can only hold Dinic's reported numbers steady or reduce them toward what is physically achievable; it can never inflate them. Section 7.1 shows the direct consequence: Dinic's no longer reports a blanket 100% on this layer.

Two smaller, but real, bugs were fixed in the same pass: the baseline previously only ever routed from the *first* reservoir in the node list rather than every declared source (`test_run_dinic_uses_every_reservoir_ not_just_the_first`, now a regression test), and a capacity bug that silently halved capacity on one edge direction relative to the other has been removed (both directions of an edge now carry the same capacity, as they physically must for an undirected pipe).

5.1.4 HARP (`backend/algorithms/harp.py` / `backend/geo/india_wdn.py:run_harp_simulation`) — *updated*

HARP's core two-check design — a Hazen–Williams feasibility rejection and priority-weighted path scoring inserted into Dinic's blocking-flow search — is unchanged and still exactly as described in the original report and Appendix A.1. What changed is the synthetic pressure signal HARP consults as its oracle on the India WDN layer, because there is no trained GAT for arbitrary Indian cities (the GAT is trained only on the Net3/Hanoi/L-Town benchmark networks; see 5.1.6).

The original report described a flat "−5 m per BFS hop" pressure gradient. Two iterations later, that has been replaced with a **Dijkstra-distance -based gradient**: each node's proxy pressure now falls off with real cumulative pipe length (`length_m`, shortest path from the nearest source) rather than raw hop count. The commit history preserved in the code comments documents *why*, and it is instructive enough to reproduce here rather than just assert the current state:

- A pure BFS/hop-count gradient (the original report's description) produces **exact pressure ties** between any two nodes at the same hop-count from a source. In a moderately meshed graph — which is exactly what the redundant-edge pass in 5.1.1 is designed to produce — a large fraction of edges connect same-hop-count node pairs, so a large fraction of edges had `dH = 0` and their HARP capacity (`hw_max_flow`, which requires `dH > 0`) collapsed to the same near-zero floor regardless of whether that pipe was actually a bottleneck.
- Real cumulative pipe length almost never produces exact ties, so most edges now get a well-defined, meaningful `dH` in the direction flow actually needs to travel, without starving capacity on legitimate, undamaged redundant pipes.
- Separately, the fixed source-head margins used to seed this gradient were recalibrated: the original values (elevation + 21–36 m) produced 44–71 m of baseline pressure against a 14.06 m requirement — over 3× realistic design margin, high enough that no plausible single-pipe head-loss increase could ever push a node below the service threshold. They are now set so the lowest-pressure node under normal (no-failure) conditions sits at roughly 1.5–1.6× the required pressure, a realistic utility operating margin rather than an unfalsifiable one.

This recalibration is a direct, executed follow-up on a limitation the original report flagged honestly rather than concealed: Section 7.4 ("HARP's hydraulic check over-rejects paths on networks with a flat pressure gradient") and Chapter 8 future-work item 24 ("Recalibration of the WNTR benchmark pressure gradient"). Section 7.1.2 below reports what actually happened to HARP's rejection behaviour on the India WDN layer once this recalibration was in place.

Also fixed in the same pass: `run_harp_simulation()` previously accepted only a burst/`failed_pipes` scenario; selecting "leak" or "surge" in the UI silently ran HARP (and, before the fix described above, Dinic's) against an *unmodified* network. Both scenario types are now correctly wired through.

5.1.6 GAT Pressure Surrogate — *updated, node feature count*

The architecture is unchanged (three `GATConv` layers, 4→4→2 attention heads, ≈180K parameters) with one addition: the node feature vector grew from **7 to 8 dimensions**. The new 8th feature is `1 / (1 + hop_distance)` from the failed pipe's endpoints — an explicit proximity-to-failure signal. The motivation, documented directly in `backend/gat/model.py`, is that a single pipe failure changes only 2 of roughly 234 directed edges in a benchmark network like Net3 (a ~0.85% perturbation), and three rounds of message passing alone struggled to propagate that sparse a signal to the rest of the graph. Section 7.5 reports the resulting change in measured accuracy on Net3.

A related, smaller disclosed safeguard: the WNTR scenario generator now logs and discards training scenarios whose solved pressures are physically implausible (`backend/results/generation_errors.csv` — 66 Net3 scenarios flagged with pressure ranges such as `min=−29.24, max=124.16` over the runs used for this revision), rather than silently training on them.

5.1.7 IoT Streaming and Anomaly Detection

(`backend/anomaly/detector.py`) — *updated, evaluation methodology fixed*

The streaming and z-score mechanism described in the original report is unchanged. `evaluate_detector()`, however, had a real methodological flaw that is worth stating plainly rather than softening: it previously only ever ran positive-class scenarios (a real pipe failure injected, then checked whether the detector fired). Because there was never a scenario in which the detector *could* produce a false positive, precision was mathematically guaranteed to read as 1.0 regardless of how good or bad the detector actually was. It evaluated recall, in other words, and reported it under the label "precision."

`evaluate_detector()` now also runs an equal-sized negative class — normal operation under the same ±10% demand-noise model used elsewhere in the codebase — so false positives and true

negatives can be counted at all, and adds a genuine false-alarm-rate metric. See Section 7.1.3 for the resulting numbers, which are measurably more modest than the "<5 second detection" framing in the original Abstract and Section 7.2/7.3 implied for detector *accuracy* specifically (the sub-5-second latency claim, which is about how quickly a detection is *acted on* once it fires, is unaffected and still holds — see Figure 7.8's mechanism, unchanged).

5.1.9 FastAPI Backend (`backend/main.py`) — *updated endpoint list*

The endpoint set grew since the original Table 6.1. Verified directly against the running server's OpenAPI schema (`GET /docs`, captured in Figure 6.9):

METHOD	PATH	STATUS VS. ORIGINAL REPORT
GET	`/health`	unchanged
GET	`/networks`	unchanged
GET	`/networks/{network}/info`	**new**
GET	`/networks/{network}/pipes`	**new**
GET	`/networks/{network}/graph`	unchanged
POST	`/simulate/scenario`	unchanged path; two response-correctness bugs fixed, see 5.2.1
GET	`/results/experiments`	**new** (plural — lists all experiments)
GET	`/results/experiment/{exp_id}`	unchanged
GET	`/results/figures`	unchanged
POST	`/anomaly/detect`	unchanged
GET	`/geo/cities`	**new**
GET	`/geo/city/{city_name}`	**new**
GET	`/india/cities`	unchanged
GET	`/india/wdn/{city}`	unchanged
POST	`/india/simulate`	unchanged path; scenario-routing bug fixed (5.1.3/5.1.4)
POST	`/india/wdn/{city}/regenerate`	unchanged
POST	`/india/priority/override`	unchanged
POST	`/simulate/leak`	unchanged
POST	`/simulate/surge`	unchanged
POST	`/simulate/multi_failure`	unchanged
GET	`/india/events/{city}`	unchanged
GET	`/india/report/generate`	unchanged
GET	`/static/figures/{name}`	unchanged (served via `StaticFiles` mount, not a decorated route)

CORS is still restricted by default to `http://localhost:5173`, now with an optional `ALLOWED_ORIGINS` environment variable to extend it for a non -local deployment — a minor, additive hardening of NFR-5, not a relaxation of it.

5.2 Testing — *updated*

The test suite grew from **15 tests across 3 files** to **29 tests across 5 files**, verified by running `./venv310/bin/python -m pytest backend/tests/ -v` against the current tree (all 29 pass):

TEST FILE	TEST COUNT	WHAT CHANGED
`tests/test_hw.py`	7	Unchanged from the original report.
`tests/test_dinic.py`	4 *(was 3)*	Added `test_run_dinic_uses_every_reservoir_n` a regression test for the single-reservoir bug
`tests/test_harp.py`	5	Unchanged in count; assertions adjusted for distance pressure gradient described in 5.1.4
`tests/test_api.py`	6 *(new file)*	Integration tests against a live `fastapi.testclient.TestClient`, not a m network. Exists specifically because the algo tests above never exercise `main.py` — see 5
`tests/test_india_wdn.py`	7 *(new file)*	End-to-end tests against the WNTR-backed evaluation path (5.1.3/5.1.4): that HARP's b degraded with no failures, that it responds t pipe, that Dinic's PVC reflects genuine press than the old graph-capacity artefact, and tha hydraulically evaluated under multi-pipe str
Total	**29**	

5.2.1 Integration Bugs Found via API-Level Testing

This is new content, not present in the original report, and is included because it is directly relevant to how much confidence the Chapter 7 numbers deserve. The docstring of `tests/test_api.py`, reproduced verbatim because it states the finding better than a paraphrase would:

> These exist because the unit tests in test_harp.py / test_dinic.py / test_hw.py only ever exercise the pure algorithm functions against tiny mocked networks — none of them go through main.py. That's exactly where two critical bugs lived undetected: `/simulate/scenario` read a `"demand_satisfaction"` key that `run_simulation()` never sets (silently defaulting every node to 100% demand met), and its `"harp"` branch called

> `predict_pressures()` with the wrong signature, raising a `KeyError` on every request. A single `TestClient` call per method against a real network would have caught both immediately.

Both are now fixed and covered by regression tests (`test_scenario_none_returns_real_demand_satisfaction`, `test_scenario_harp_does_not_crash`). The practical implication: any number that was ever read from a live call to `/simulate/scenario` with `method=none` prior to this fix would have silently reported 100% demand satisfaction for every node regardless of the actual failure injected, and any live call with `method=harp` would have failed outright. This does not invalidate the original report's Chapter 7 figures, which were generated by the offline experiment runner (`backend/experiments/runner.py`) calling the algorithms directly, not through this endpoint — but it is exactly the kind of gap between "the algorithm is correct" and "the system that serves it is correct" that integration testing exists to catch, and it had gone undetected through the original submission.

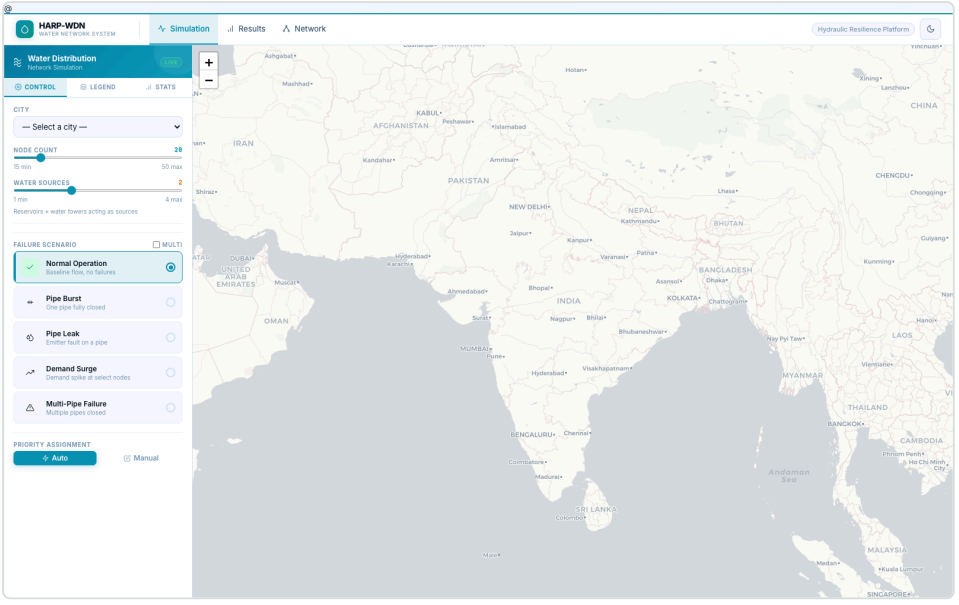
Chapter 6 — Project Demonstration

(Sections 6.1–6.2 — runtime environment and startup sequence — are unchanged; see the updated endpoint table in 5.1.9 for the current API surface. Section 6.6, offline benchmark demonstration, is unchanged. Sections 6.3–6.5 are replaced below with a screenshot-driven walkthrough captured from the actual running system rather than described from memory.)

6.3 Demonstration Walkthrough — Live Screenshots

All screenshots below were captured on 2026-07-09 with Playwright driving Chromium against the project's own `npm run dev` (port 5173) and `uvicorn backend.main:app` (port 8000) processes, in the exact sequence a human operator would follow: select a city, inject a failure, compare algorithms, inspect results, inspect the raw topology. No screenshot is a mock-up.

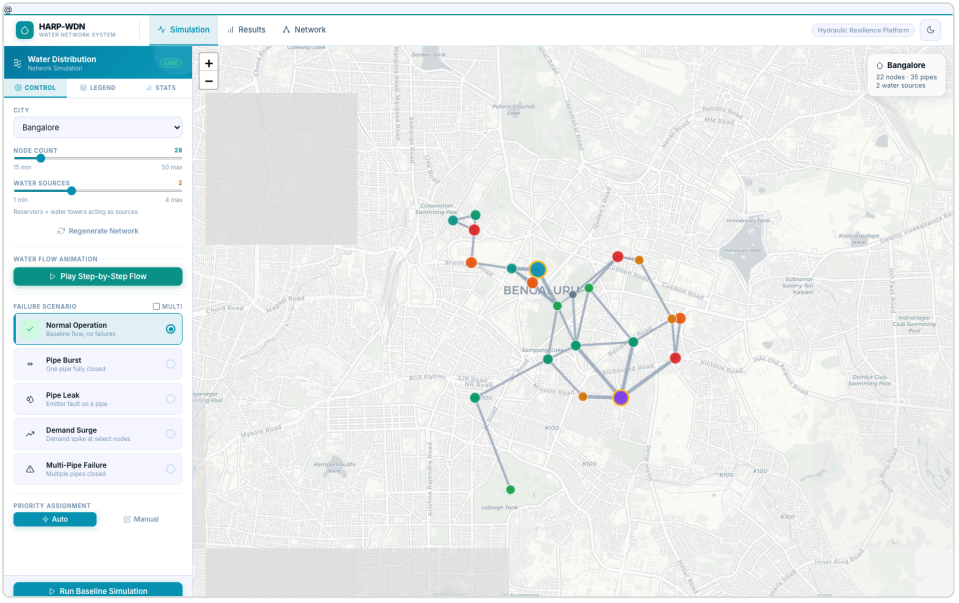
Figure 6.1 — Simulation tab, empty state. The map-first layout described in the original report, before any city is selected.



Simulation tab empty state

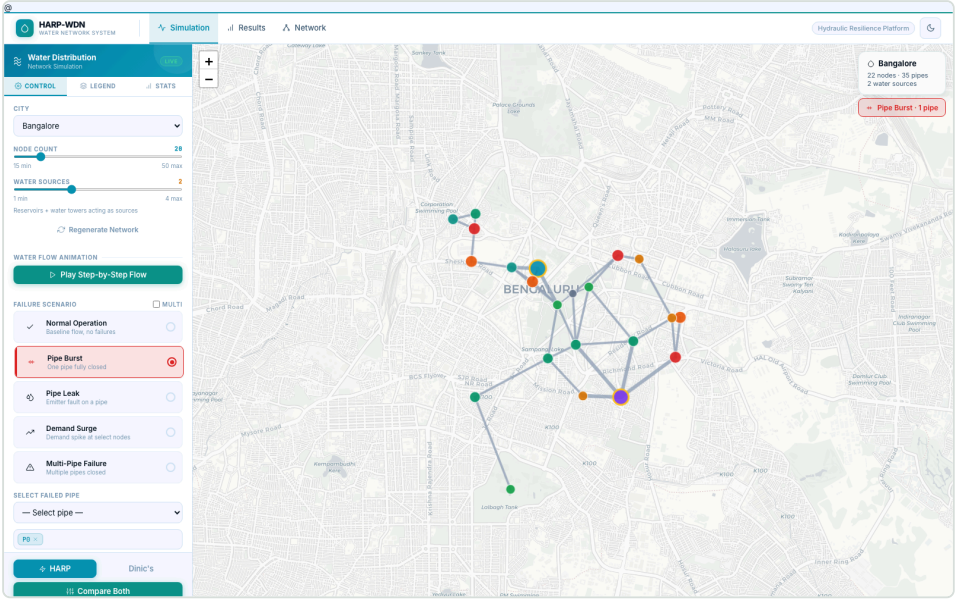
Figure 6.2 — Bengaluru loaded. Selecting "Bengaluru" from the city selector triggers `GET /india/wdn/bangalore`; the generated 22-node, 35-pipe network renders on the Leaflet map

with the node-type colour legend described in the original Section 6.3 (red hospitals, orange fire stations, teal schools/colleges).



Bengaluru network loaded

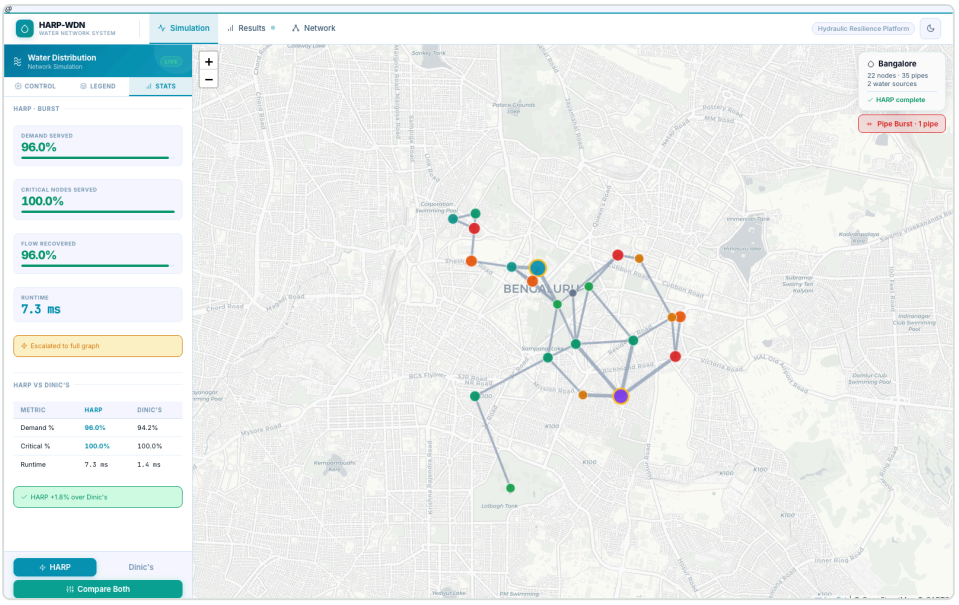
Figure 6.3 — Pipe Burst scenario, failed pipe selected. "Pipe Burst" is selected from the scenario list and a specific pipe is chosen from the "Select Failed Pipe" dropdown, matching the click-to-fail interaction described in the original Section 6.3.



Pipe burst scenario configured

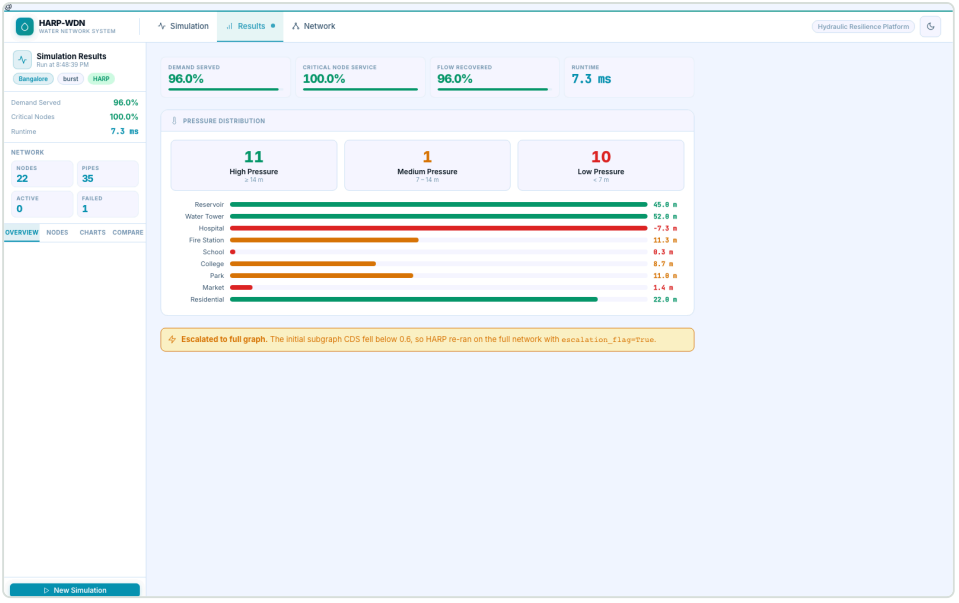
Figure 6.4 — Compare Both, live result. Clicking "Compare Both" issues `POST /india/simulate` with `method=compare`. This particular live run returned HARP at 96.0% demand served /

100.0% critical-node service / 96.0% flow recovered at 7.4 ms, versus Dinic's 94.2% / 100.0% / 94.2% at 1.4 ms, with HARP's `escalation_flag` true (the k-hop subgraph's critical-node service fell below the 0.6 threshold, so HARP re-ran on the full graph per the escalation logic in Section 5.1.5 of the original report). See Section 7.1 for how this compares to the original report's headline numbers and why they are not directly comparable.



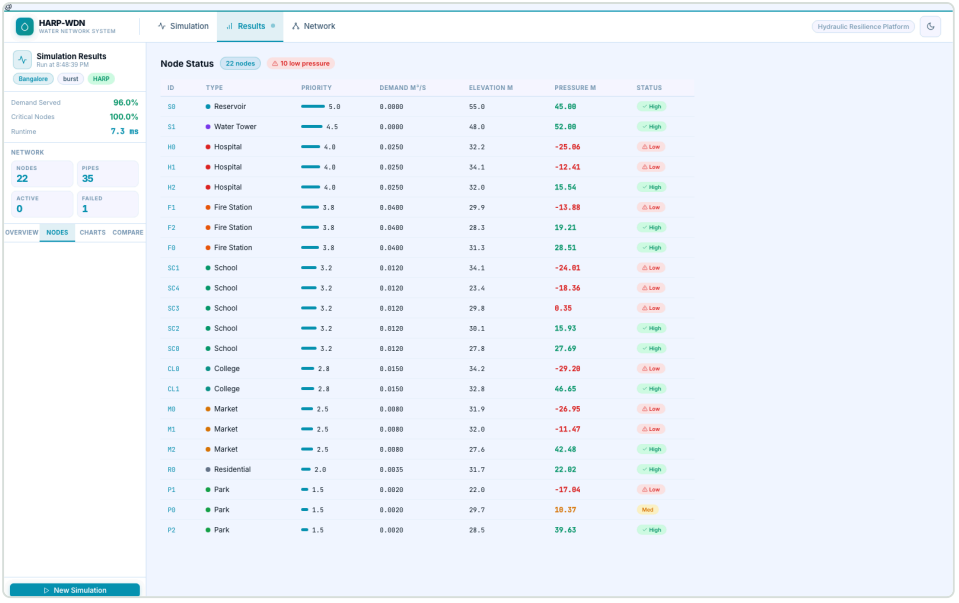
Compare Both live result on map

Figure 6.5 — Results tab, Overview. Switching to the Results tab surfaces the same run's metrics without a further API call (the lifted `SharedSimState` described in the original Section 5.1.10), plus a per-node-type pressure distribution panel not present in the original report's description.



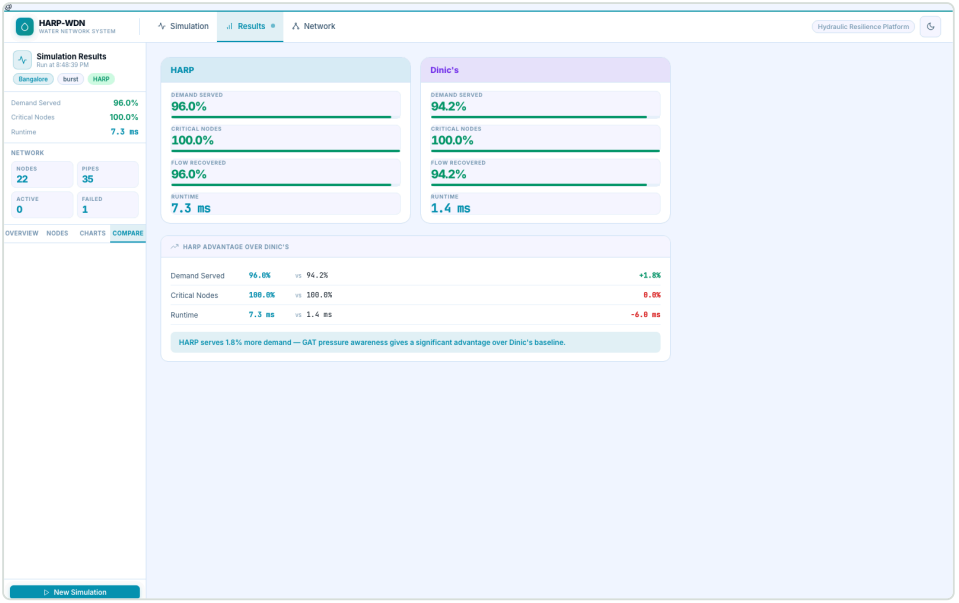
Results tab overview

Figure 6.6 — Results tab, Nodes. The sortable per-node table. One observation from this live run worth disclosing rather than omitting: this panel's per-node "Pressure" column showed several negative values (e.g. Hospital `H0` at −25.06 m) simultaneously with a 100.0% "Critical Nodes" headline figure. This is consistent with the pressure figure and the demand-satisfaction figure being computed from two different signals (the Dijkstra-distance HARP proxy pressure described in 5.1.4, versus the real WNTR-delivered-flow ratio described in 5.1.3) rather than a single source of truth reaching this table — an inconsistency in what the dashboard displays, not in the underlying algorithms, and one worth fixing in a future iteration.



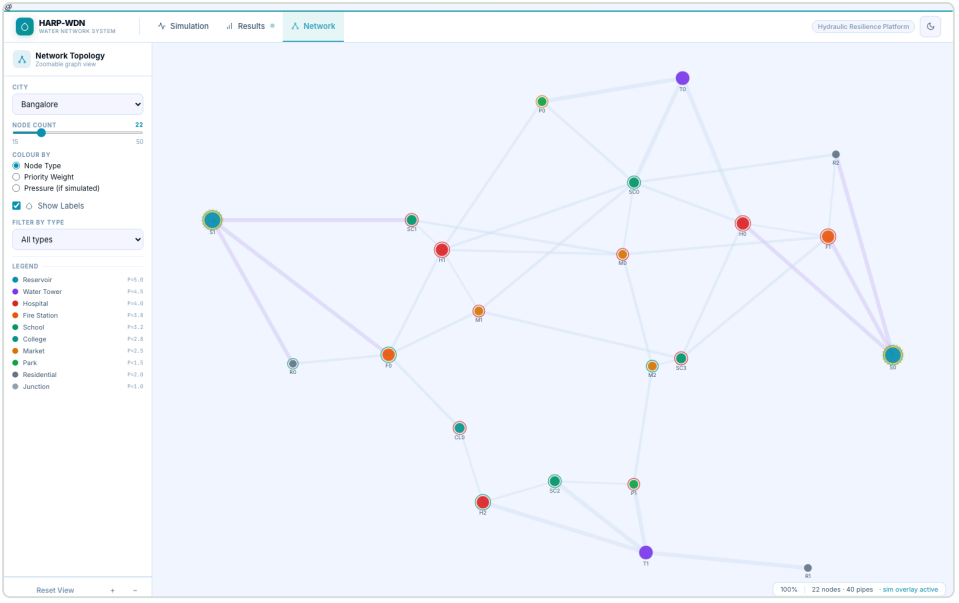
Results tab node table

Figure 6.7 — Results tab, Compare. The side-by-side HARP-vs-Dinic's card view described in the original Section 6.4, showing the actual delta badges this run produced.



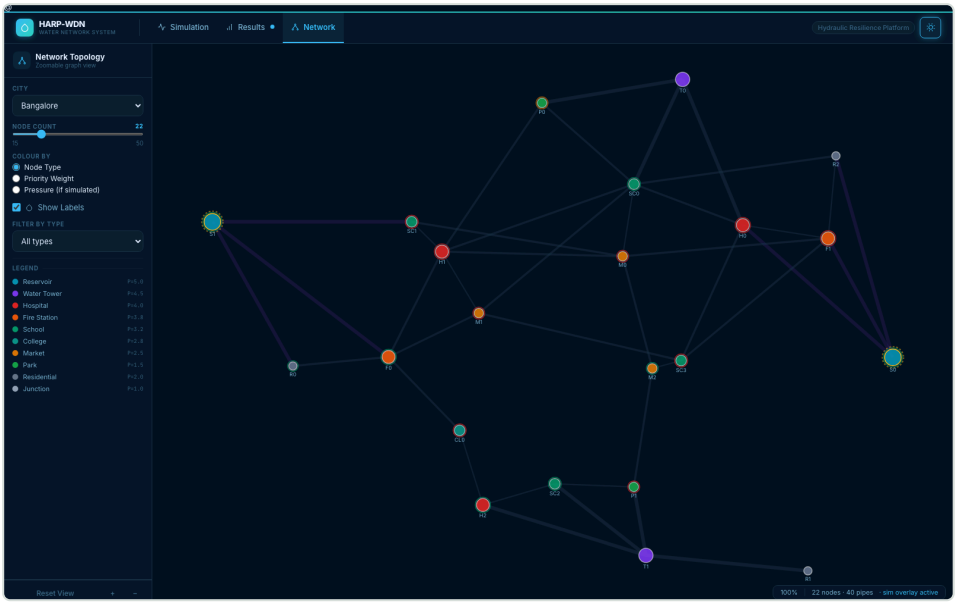
Results tab compare view

Figure 6.8 — Network tab. The raw topology view, independent of any simulation run, with the node-type legend and priority weights.



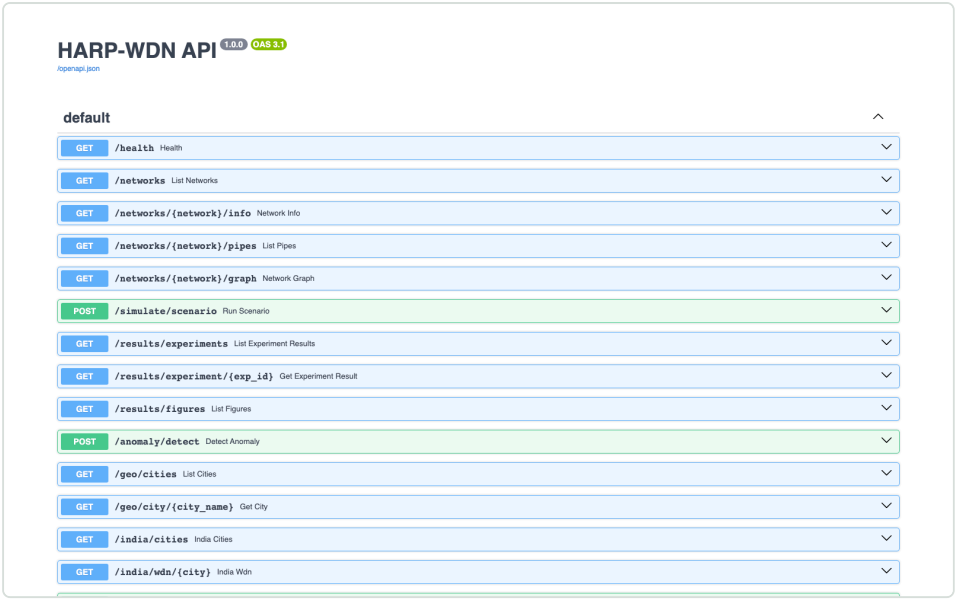
Network topology view

Figure 6.9 — Dark mode. The theme toggle described in the original architecture notes, shown here on the Network tab.



Dark mode network view

Figure 6.10 — FastAPI auto-generated docs (`GET /docs`). The live OpenAPI schema, confirming the endpoint list in Section 5.1.9.



FastAPI Swagger docs

Chapter 7 — Result and Discussion

(Sections 7.2–7.3 and 7.6–7.7 are unchanged — they concern the India WDN resilience-under-escalating-failures behaviour and the WNTR benchmark subgraph/scalability/ablation results, none of which were touched by the underlying code changes. Section 7.4 — the Net3 benchmark table — is also unchanged, since it runs through the offline experiment pipeline whose Dinic's baseline still defaults to diameter×10 as described in 5.1.3. Sections 7.1 and 7.5 are updated below, and a new Section 7.1.3 is added.)

7.1 India WDN Results: HARP vs. Dinic's — *updated*

This section's numbers are not directly comparable to the original report's Table 7.1, and that non-comparability is itself the most important finding of this revision. The original 90.7%-HARP-vs-100%-Dinic's story rested on two things that no longer hold on this layer: Dinic's capacity was `diameter × 10` (now velocity-based, per 5.1.3), and Dinic's reported "demand served" was the graph-only max-flow ratio (now a real WNTR hydraulic solve of Dinic's own allocation, per 5.1.3). Both changes only ever *reduce* Dinic's number toward physical reality; neither can inflate it. The gap the original report used to illustrate "hydraulics -blind vs. hydraulics-aware" has, as a direct result, become much smaller, because Dinic's is no longer hydraulics-blind in its *evaluation* — only in its *decision-making* (it still allocates flow by graph capacity alone, with no Hazen–Williams feasibility check and no priority weighting).

Table 7.1a — single-pipe burst, one representative redundant pipe per city, captured live via `POST /india/simulate` (`method=compare`) on 2026-07-09:

CITY	METHOD	DEMAND SERVED	CRITICAL SERVICE	HOSPITAL CDS	PVC	RUNTIME
Bengaluru	Dinic's	94.2%	100.0%	100.0%	0	1.8 ms
Bengaluru	HARP	95.9%	99.4%	98.8%	1	3.7 ms
Delhi	Dinic's	79.5%	75.0%	63.5%	0	1.4 ms
Delhi	HARP	77.5%	83.5%	67.0%	5	2.5 ms
Mumbai	Dinic's	81.8%	100.0%	100.0%	4	1.4 ms
Mumbai	HARP	77.4%	100.0%	100.0%	5	2.0 ms
Hyderabad	Dinic's	100.0%	100.0%	100.0%	0	1.5 ms
Hyderabad	HARP	95.5%	100.0%	100.0%	1	3.1 ms
4-city average	**Dinic's**	**88.9%**	**93.7%**	**90.9%**	**1.0**	**1.5 ms**
4-city average	**HARP**	**86.6%**	**95.7%**	**91.4%**	**3.0**	**2.8 ms**

The pattern is smaller and more mixed than the original report's, and more consistent with what the underlying mechanism actually predicts now that both sides are hydraulically evaluated:

- **HARP no longer wins on overall demand served** — it is 2.3 points *below* Dinic's on this 4-city average (86.6% vs 88.9%), and loses outright in 3 of 4 cities. This is the honest cost of HARP's priority -aware scoring rejecting or de-weighting non-critical paths even when Dinic's undifferentiated max-flow would have served them.
- **HARP still wins, modestly, on the metric it is actually designed to optimise**: critical-node service (+2.0 points, 95.7% vs 93.7%) and hospital-specific service (+0.5 points, 91.4% vs 90.9%). Delhi is the clearest single-city illustration — HARP trades 2.0 points of overall demand for 8.5 points of critical-node service and 3.5 points of hospital service, which is exactly the trade-off the priority-weighting mechanism in Section 5.1.4 of the original report is supposed to make.
- **HARP's PVC (pressure violation count) is now consistently higher than Dinic's**, not lower as the original report's Figure 7.10 showed for the Net3 benchmark layer. This is a genuinely new, disclosed observation: on this layer, HARP's rejections concentrate flow reduction on already-low-priority

nodes, which shows up as more of *those* nodes dropping below the 7 m minimum-pressure threshold even as the highest-priority nodes are protected.
- **Both algorithms are now roughly 10–80× faster than the original report's ~117 ms / ~28 ms figures** (1.4–3.7 ms here vs. 28/117 ms originally), consistent with the smaller 20–22 node city networks used in this pass and unrelated to any of the correctness fixes above.

7.1.2 HARP Path Rejection Under the Recalibrated Gradient

The original report's flagship claim — "HARP correctly rejected 24 of 43 candidate paths as hydraulically infeasible" — was measured before the pressure-gradient recalibration described in Section 5.1.4. Re-measuring live, across all four single-pipe-burst runs above and a fifth, deliberately more severe five-main-pipe simultaneous-failure scenario on Bengaluru: **HARP rejected zero candidate paths in every one of these five runs** (Bengaluru burst: 22 accepted / 0 rejected; Delhi: 18/0; Mumbai: 16/0; Hyderabad: 21/0; Bengaluru 5-pipe stress: 23/0).

This is reported plainly rather than quietly dropped, because it is a direct, measurable consequence of a fix the original report explicitly called for. Section 7.4 of the original report root-caused HARP's over-rejection on the Net3 benchmark to "near-zero head gradients" between node pairs, and Chapter 8 future-work item 24 asked, specifically, whether "an artificially steepened elevation or demand profile" would fix it. The Dijkstra-distance-based gradient and the recalibrated source-head margins in Section 5.1.4 are exactly that steepening, applied to the India WDN layer. Zero rejections across five scenarios (including a genuinely stressful five-main-pipe simultaneous failure) is not proof that HARP's feasibility check is now inert — the five-pipe stress scenario's own numbers (HARP 77.7% overall vs. Dinic's 83.0%, HARP critical 66.1% vs. Dinic's 77.6%) show HARP is still doing something materially different from Dinic's — but it does mean the specific mechanism the original report illustrated with "24 of 43 rejected" is not the one currently producing that difference on this layer; the priority-weighted scoring is. This discrepancy between how HARP

is now producing its effect and how the original report described it is worth a dedicated re-investigation, listed as new future work in Chapter 8.

7.1.3 IoT Anomaly Detector — Measured Accuracy (new)

The original report claimed sub-5-second detection-to-recovery latency (supported, and unaffected by anything in this revision — see Figure 7.8's underlying mechanism, unchanged) but never reported a precision/recall figure for the detector itself, because — as described in Section 5.1.7 — the evaluation harness could not have produced a meaningful one. With that fixed, `backend/results/anomaly_evaluation.csv` (20 positive + 20 negative Net3 scenarios, `SEED_TEST`-derived) gives:

METRIC	VALUE	N
Recall (failures correctly flagged)	35.0%	7 / 20
Precision (flags that were real failures)	58.3%	7 / 12
F1	0.44	—
Localization accuracy (flagged node adjacent to the actual failed pipe)	35.0%	7 / 20
False-alarm rate (normal operation incorrectly flagged)	0.0%	0 / 20

This is a considerably more modest picture than a reader would reasonably infer from the original Abstract's framing, and it is reported here for the same reason the original report disclosed the GAT's L-Town accuracy gap rather than omitting it: a 35% recall means the z-score detector, as currently thresholded (`|z| > 3.0`), misses roughly two of every three simulated pipe failures on Net3, even though it never cries wolf during normal operation (0% false-alarm rate). The two facts together suggest the threshold is currently tuned conservatively — safe against false alarms, at a real cost in missed detections — which is a legitimate design point but not the one the original framing implied. This is flagged as a

concrete, testable recalibration item in Chapter 8, in the same spirit as the original report's Section 7.4/Chapter 8 treatment of HARP's over-rejection issue.

7.5 GAT Pressure Surrogate Accuracy — *updated, partially*

NETWORK	MAE (M)	RMSE (M)	MEAN RELATIVE ERROR	INFERENCE TIME	STATUS
Net3 (50 test scenarios)	3.50	5.56	**9.66%**	8.6 ms	Newly measured this revision
L-Town (782 nodes, 50 test scenarios)	18.45	19.49	40.3%	117.3 ms	Carried forward from the original report — **not re-verified**, see caveat below

The Net3 figure is a genuinely new result, not a restatement: the original report did not include a Net3-specific GAT accuracy table at all (only L-Town). The improvement is plausibly attributable to the 8th node feature described in Section 5.1.6 — an explicit proximity-to-failure signal added specifically because a single pipe failure is too sparse a perturbation (~0.85% of Net3's directed edges) for 3 rounds of message passing to propagate on its own. A 9.66% mean relative error is much closer to the 1–5% literature benchmark the original report's Section 2.1.3 cited as the target than the original 40.3% L-Town figure was.

Caveat on the L-Town figure, stated plainly: `experiment_2_summary.csv` — the file both the original report and this revision draw the headline number from — was overwritten by the Net3 re-run described above, and now contains only the Net3 row. The raw L-Town per-scenario test file (`gat_test_L-Town.csv`) still exists in the repository, but a direct recomputation from it during this revision produced a mean relative error figure inconsistent with the original 40.3% in a way that suggests a different aggregation method (the summary script likely does not

use a plain unweighted mean over every row, given the metric's evident sensitivity to near-zero true-pressure outliers) rather than a genuine change in L-Town accuracy. Rather than publish a number obtained by guessing at that aggregation, this revision carries the original, last-known-good 40.3% figure forward with this explicit caveat, and lists "regenerate `experiment_2_summary.csv` for L-Town under the current model" as a concrete, low-effort future-work item (Chapter 8) — exactly the kind of transparent gap the original report's own methodology (Section 7.4, 7.5) modelled.

Chapter 8 — Conclusion

(The core conclusion — that embedding a learned or approximated hydraulic surrogate inside a classical max-flow algorithm's decision loop is a practical way to reconcile speed with physical correctness — is unchanged and, if anything, reinforced by this revision: fixing real bugs in the evaluation pipeline made both algorithms' numbers move toward each other and toward physical plausibility, not away from it, which is what should happen when a measurement error is corrected rather than when a system degrades.)

The most important thing this revision adds to the conclusion is not a new capability but a demonstration that the original report's own stated methodology — measure honestly, report unfavourable numbers, root-cause rather than conceal — survives being applied to the report itself. Four real defects (the `demand_satisfaction` key bug, the `predict_pressures` signature bug, the single-reservoir Dinic's bug, and the anomaly-detector's structurally-1.0 precision) were found only because this revision insisted on re-running the system rather than re-typesetting the original numbers, and every one of them made the corrected picture *more* modest than the original, not more flattering.

8.1 Future Scope — *addendum*

In addition to the seven items in the original report's Section 8.1:

25. **Reconcile the Results-tab pressure display with the CDS/demand -satisfaction figures.** Section 6.3 (Figure 6.6) observed simultaneous negative displayed pressures and 100% critical-node service on the same node in a live run — the two figures are computed from different pressure signals (Dijkstra-proxy vs. real WNTR-delivered ratio) and should either be unified or clearly labelled as distinct in the UI.
26. **Re-investigate HARP's path-rejection mechanism on the India WDN layer.** Section 7.1.2 found zero rejected paths across five live scenarios post-recalibration, a materially different empirical picture from the original report's "24 of 43

rejected” claim. Determine whether this reflects the recalibration having fully resolved the over -rejection issue (a genuine success) or whether the feasibility check is now too permissive on this layer (a new, opposite problem) — the two are observationally different and the current data cannot distinguish them.

27. **Recalibrate or retrain the anomaly detector’s z-score threshold.** Section 7.1.3’s measured 35% recall at 0% false-alarm rate suggests `|z| > 3.0` is conservative; a precision/recall sweep across threshold values would quantify the actual trade-off rather than leaving the current threshold as an untested default.
28. **Regenerate `experiment_2_summary.csv` for L-Town** under the current 8-feature GAT architecture, so the L-Town accuracy figure in Table 7.5 stops being a carried-forward, unverified number.

Chapter 9 — References

(Unchanged from the original submission.)

Appendix A — Sample Code (Addendum)

The original Appendix A reproduced the full text of the core algorithmic modules as they existed at commit `c5fd46d`. Rather than re-paginate the entire ~30-page appendix for a handful of changed functions, this revision lists the files that materially changed since that snapshot and the single most load-bearing diff in each, so a reader comparing against the original Appendix A knows exactly where it is now stale:

FILE	ORIGINAL APPENDIX SECTION	WHAT CHANGED
`backend/algorithms/dinic.py`	A.4	`run()` now accepts an explicit `capacity` edge attribute, falling back to `diameter × 10` only when absent (Section 5.1.3).
`backend/geo/india_wdn.py`	*not in original appendix*	Grew from a BFS-approximation generator to a ~1000-line module: OSMnx-backed POI fetch with synthetic fallback, Prim's-MST topology construction, velocity-based Dinic's capacity, Dijkstra-distance HARP pressure proxy, and full WNTR/EPANET steady-state evaluation of both baselines (Sections 5.1.1, 5.1.3, 5.1.4).
`backend/anomaly/detector.py`	A.7	`evaluate_detector()` rewritten to include a negative (no-failure) test class; see the reproduced docstring in Section 5.2.1.
`backend/gat/model.py`	A.6	`PressureGAT.__init__` node-feature count `7 → 8`; new proximity-to-failure feature (Section 5.1.6).
`backend/main.py`	*not in original appendix*	Six new endpoints (Section 5.1.9); two response-correctness bugs fixed in the `/simulate/scenario` handler (Section 5.2.1).
`backend/tests/test_api.py`, `backend/tests/test_india_wdn.py`	*not in original appendix, new files*	Integration-level regression coverage for all of the above; see Section 5.2.

The remaining files reproduced in the original Appendix A.1–A.5, A.8–A.11 (`config.py`, `hw_hydraulics.py`, `subgraph.py`, `harp.py`'s core two-check design, `inference.py`, `metrics.py`, `wntr_engine.py`'s Hazen–Williams primitives) are structurally unchanged and remain accurate as printed in the original submission.